

The Elements of Prompting — Field Guide

What every template is made of: each element defined — mechanism, strong vs. weak fills, and the failure it prevents.

Why elements instead of “good prompts”: output quality swings up to 76 accuracy points from formatting alone (Sclar et al., ICLR 2024). You don’t reword your way to a reliable prompt — you find which element failed and fix that one. Learn the elements and you can diagnose any bad output, and build templates of your own instead of collecting other people’s.

THE DIAGNOSIS GRID

The output is...	The element that failed
the wrong altitude, tone, or posture	Role
an answer to a different (or vaguer) question	Task
generically right, specifically wrong for us	Context
right content, unusable shape or length	Format
not matching the standard in your head	Examples
confidently invented	The Out (missing)
sprawling past what you asked	The Stop (missing)

PART 1 — GENERATIVE ELEMENTS

A prompt that writes: five elements, two safety valves

A generative prompt commissions one piece of text you will read and act on. Everything you need to control fits in five elements — plus two one-sentence safety valves that shut off the two signature failures.

1 · ROLE — who is answering

Definition. The identity, expertise posture, and — critically — the behavioral commitments you assign before the task begins.

Mechanism. An LLM completes documents by predicting what plausibly comes next. The role selects which region of everything it has read the completion is drawn from: vocabulary, caution level, what gets foregrounded. “You are a careful analyst” makes analyst-style continuations more probable — it does not add analyst knowledge.

Evidence & nuance. Personas do not improve factual accuracy (162-persona study, Zheng et al. 2024) — “you are a genius mathematician” doesn’t fix arithmetic. They do reliably shift tone, framing, and foregrounded knowledge (Salewski et al. 2023). Use Role for how it answers; never as a substitute for checking what it answers. Behaviors beat titles because behaviors are auditable in the output.

Weak: “You are a senior data scientist.” — a credential; produces nothing checkable.

Strong: “You are a careful analyst. You compute every number by running code, you state n for every proportion, you never drop data silently, and you say ‘the data can’t answer that’ when it can’t.” — four behaviors, each auditable.

Failure it prevents: the generic-assistant voice — mid-distribution, hedge-everything output.

2 · TASK — what, for whom, to what end

Definition. The single clear ask: a verb + object + audience + success criterion.

Mechanism. The most load-bearing words in the prompt — they define the target the completion optimizes toward. Vendors agree to the point of redundancy: Google calls the task “the most important component”; for Microsoft the Goal is the only required part.

Nuance. One task per prompt; if several, number them in priority order and add the Stop. A question beats an instruction because it carries its own completion test — either it got answered or it didn't. Include the audience and the outcome: "the reader should be able to decide X."

Weak: "Analyze the returns data." — a topic. Topics generate exploration.

Strong: "Answer: what is the return rate by site for Q2 versus the 2.0% target, and which three reason codes drive 80% of returns?" — questions generate answers.

Failure it prevents: wandering output — ten observations and no answer.

3 · CONTEXT — what the model cannot know

Definition. Everything true in your world that isn't in the training data: raw material, definitions, constraints, history, quirks — and the reasons behind your rules.

Mechanism. The model fills every information gap with the most statistically plausible filler. Context replaces plausible-in-general with true-for-you — this is the anti-hallucination element; most "lies" are really unsupplied context.

What goes in it. The raw material itself (never assume it knows your documents) · a glossary for terms with local meaning, used exactly · constraints and sensitivities · known quirks each with the rule to apply · and the why behind non-obvious rules — "this will be read aloud, so no ellipses" measurably outperforms "no ellipses"; models generalize from reasons.

Discipline. Delimit context from instructions (tags, fences, headers) so material to work on is never confused with orders to follow — also your first defense when the material itself contains instruction-like text.

Weak: "Use our standard definitions." — the model's "standard" is the internet's average.

Strong: "Glossary — use exactly: Return rate = returned units ÷ shipped units (shipped_q2.csv). Known quirk: the export has a totals row at the bottom — exclude it and say so."

Failure it prevents: generically-correct, specifically-wrong output — the plausible number with your company's name on it.

4 · FORMAT — the output contract

Definition. The required shape of the response: structure, length, tone, medium, what to include and omit.

Mechanism. Models imitate structure more readily than they obey descriptions of it — so Format works best stated positively ("respond in flowing prose paragraphs" beats "don't use markdown") and best of all demonstrated (see Examples).

Nuance. Give a numeric length cap with a priority rule ("≤120 words — cut content before quality"); adjectives like "short" are unenforceable. On genuinely hard reasoning, heavy format constraints can tax accuracy — let it reason free-form first, then format the answer in a second step.

Weak: "Keep it short and professional."

Strong: "Max 120 words. Structure: the ask in sentence one; two supporting facts with numbers; the deadline. Nothing else."

Failure it prevents: the 3x-too-long draft; the buried answer; output you must re-shape by hand.

5 · EXAMPLES — show, don't describe

Definition. 3-5 realistic demonstrations of the task done right, placed in the prompt.

Mechanism. In-context learning (Brown et al. 2020 — the discovery that founded prompt engineering): the model infers the pattern from demonstrations without retraining. The strongest steering signal that exists for format, tone, and edge-case handling.

Nuance. Diversity is the point: include the awkward case (missing field, borderline grade, angry customer) — the model handles edges exactly as your examples do. Wrap them in tags so they can't be mistaken for live input. 2026: on thinking models, start zero-shot; add examples only when format drifts (few-shot measurably degrades some reasoning models).

Weak: three examples that are the same happy case reworded.

Strong: one typical case + one edge case + one reject/escalate case — now the model knows the boundaries, not just the center.

Failure it prevents: the model guessing your standard from the internet's average; format drift across runs.

THE TWO SAFETY VALVES

THE OUT — permission to not know. “If the document doesn't say, say so.” Models are trained on benchmarks that reward guessing over abstaining (OpenAI, 2025); the Out re-opens the abstain option and drastically cuts invented answers. Without it you have implicitly ordered the model to always produce something.

THE STOP — the scope boundary. “Answer these three questions, then stop — do not go exploring.” Converts open-ended capability into bounded work. In agentic prompts it grows up to become gates and autonomy rules.

PART 2 — AGENTIC ELEMENTS

A prompt that works: the twelve blocks of the mission brief

An agentic prompt commissions a job, not a text: the agent plans, acts through tools, checks results, and iterates — mostly while you are not watching. Every generative element still applies. The additional blocks exist for one reason: text that fails costs you a re-prompt; actions that fail change the world — files overwritten, emails sent, wrong numbers published. So the agentic elements govern conduct: where the agent may act, how it must verify itself, when it must stop and ask, and how it proves what it did. (Template A1; worked version A2.)

1 · <role> — the standing behavioral contract

Same element as generative Role, with higher stakes: in chat it shapes one answer you're about to read; in an agent it biases hundreds of unsupervised decisions. Write it as the behaviors you'd want if you couldn't check any single step: "never invents numbers, logs every transformation with a row count, prefers reproducible scripts over manual edits." **Prevents:** an agent that improvises its professional standards mid-run.

2 · <mission> — the goal and the definition of done

The agentic Task: one goal, the decision it feeds, audience, deadline — plus the line chat never needs: "done looks like: ...". An agent runs a loop; the mission is its exit condition. Vendor grounding: give the agent "a check it can run... without one, 'looks done' is the only signal available." **Prevents:** the agent deciding for itself what finished means — plausible activity, indefinitely.

3 · <context> — tribal knowledge, written down

Same element as generative Context; the agentic difference is coverage pressure — an agent touches your domain at every step, so every unstated quirk gets stepped on. The glossary becomes a controlled vocabulary (it will otherwise compute three subtly different "yields"); each quirk carries its rule ("Parks records PN 668518; it means 666518 — map it"). **Prevents:** silent invented fixes — the most expensive agent error, because they look like diligence.

4 · <environment> — the workspace contract

New in kind. Where the agent may read, where it must write, what tools exist, what is forbidden. Load-bearing distinctions: inputs are read-only · outputs are versioned (never overwrite what a human opened) · scratch is regenerable. Name forbidden actions explicitly (no network calls, no emails, no writes outside the folder). Least privilege as prose — enforce with sandboxing too: the prompt is policy, the sandbox is physics. **Prevents:** the agent "helpfully" editing your master workbook.

5 · <inputs> — the source register

New in kind. One block per source: location, exact format, grain ("one row = one inspected unit" vs "one lot with counts" — the line that determines every denominator downstream), primary key, coverage, owner, trust level (authoritative / derived / manual entry), known issues. Cardinal rule: a fact you can't establish is written **UNKNOWN** and raised as an open item — never guessed. **Prevents:** the agent inferring your schema — where most wrong numbers are born.

6 · <plan> — bounded method

The steps in order, plus two lists chat prompts don't carry: methods allowed (boring, reproducible) and methods not allowed without asking (dropping data, causal claims, model fits, new dependencies). Specific enough to prevent improvised rigor, open enough to adapt to what it finds. **Prevents:** creative methodology at step 14 of 20, discovered only at review.

7 · <checks> — machine-evaluable verification

New in kind — the honesty engine of the whole brief. Verification the agent can actually run, split by consequence: HARD checks stop the run (totals reconcile to a trusted number; pass + fail = inspected; no duplicate keys); SOFT checks flag and continue (null spikes, small n, metric moved >X vs last cycle). Two disciplines: write checks as equalities and set-memberships, not aspirations ("check data quality" is not a check); and always include one reconciliation anchor — a number you already trust from outside the run. Without an anchor, every other check can pass on the wrong data. **Prevents:** plausible-but-wrong deliverables — the agent's version of hallucination.

8 · <outputs> — the deliverable contract and audit trail

The agentic Format: exact filenames (with date suffixes), exact locations, and the three logs that make a run auditable — CHANGES.md (what changed, when, why) · FINDINGS.md (claims → evidence → confidence) · OPEN_ITEMS.md (assumptions and questions, each with owner and status). A colleague should be able to find, understand, and regenerate everything from the folder alone. **Prevents:** results you can't locate and conclusions you can't trace six weeks later.

9 · <process> — phases and human gates

New in kind. The job cut into phases with human checkpoints at the cheap-error points: Gate 1 confirms definitions before any data is touched · Gate 2 reviews reconciliation before analysis builds on the numbers · Gate 3 approves headlines as plain text before anything renders or ships. A definition error caught at Gate 1 costs a sentence; the same error after the deck circulated costs a retraction. **Prevents:** compounding — hours of competent work on a wrong foundation.

10 · autonomy rules — when to proceed, when to stop (inside)

The agentic Stop, matured. Between gates the agent proceeds without asking; it must stop when: a HARD check fails · an input isn't as described · a definition is ambiguous · an action is hard to reverse (delete, send, publish, overwrite) · a previously published number would change. Both halves matter: without "proceed" you built a slow chatbot; without "stop" you built a liability. General principle: reversible → act; irreversible → ask.

11 · rules + quality_bar — the invariants, and done-as-a-checklist

<rules> are the non-negotiables kept short and absolute so they survive a long context: never invent or back-fill a number · sources are immutable · every transformation logged with a row count · assumptions stated as assumptions. Rules differ from checks: checks are evaluated at defined points; rules hold everywhere. **Prevents:** standards drift in hour three.

<quality_bar> is the definition of done made auditable: every placeholder resolved or logged · all HARD checks pass, every SOFT flag noted · every deliverable re-opened and verified · logs written · every presented number traceable to a source cell. Checks verify the work mid-run; the quality bar verifies completeness at the end. **Prevents:** "done" meaning "I stopped."

12 · <reporting> — the interface back to you

The agentic Format for the message, fixed in shape: status (done / stopped at gate n / blocked) · headline answer with the number · deliverable paths · checks summary · open items needing a human · risks. Under 25 lines; evidence, not assertions — the test it ran and what it returned, not “everything went well.” Details live in the files. **Prevents:** a narrative essay where a status should be.

PART 3 — THE MAPPING

The agentic brief is the generative anatomy, grown up to survive autonomy

Generative element	Agentic descendant(s)	What was added, and why
Role	<role>	persists across unsupervised decisions → behaviors only
Task	<mission>	+ definition of done — the loop needs an exit condition
Context	<context> + <inputs>	+ per-source register with grain & trust — agents touch every source repeatedly
Format	<outputs> + <reporting>	+ audit logs and a fixed status shape — actions must be traceable
Examples	the filled brief (A2)	a worked run is the agentic few-shot
The Out	UNKNOWN → open items	not-knowing becomes a logged, owned item
The Stop	gates + autonomy rules	scope control becomes checkpointed process
(none)	<environment> <plan> <checks> <rules> <quality_bar>	new in kind — conduct, method, verification, invariants, completeness: things text-only prompts never needed

When to switch modes: if the work involves files, tools, multiple steps, or a deliverable produced while you're not watching — write the brief. If you'll read the output and act on it yourself — the five elements and two valves are enough.

PART 4 — BUILDING YOUR OWN TEMPLATES

A template is elements + reasons, assembled against failure modes

1. Name the task's three most expensive failure modes (wrong denominator? invented facts? unusable format? irreversible action?).
2. Pick the element that owns each failure — the diagnosis grid on page 1; reversibility → autonomy rules.
3. Write those elements strong; keep the rest light. A template where every element is maximal is a template nobody fills in.
4. Add one filled example — the gold standard teaches faster than the instructions do.
5. Test on 3-5 real cases (one edge case), version it, name an owner, add it to the library.